Lab 4: Linear Regression

Step-by-Step Instructions:

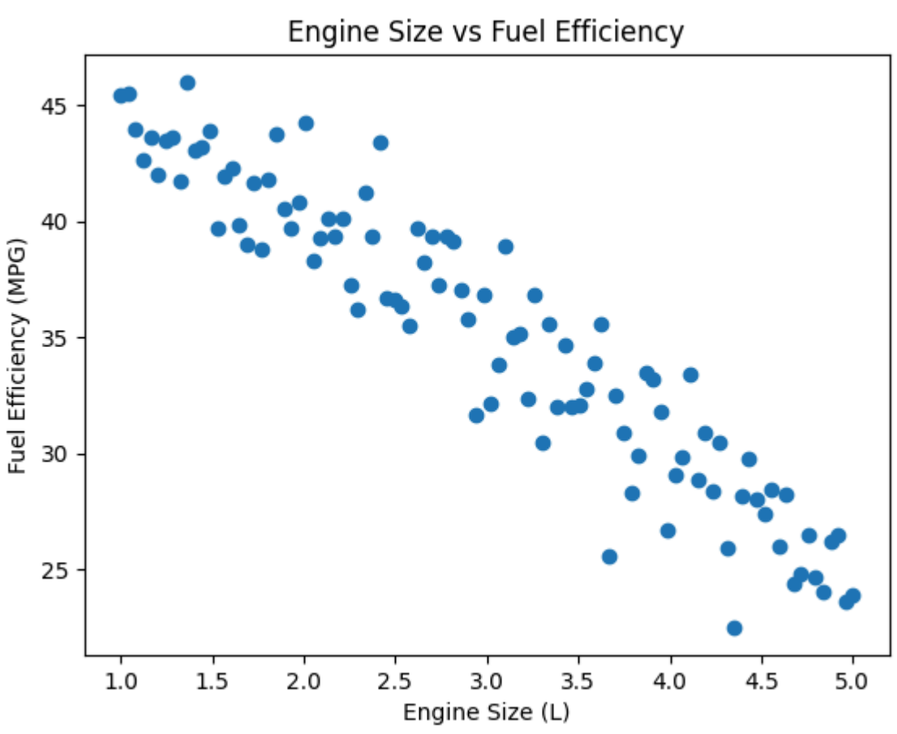
In [11]:
```python
# In this lab we'll walk through a simple linear regression example using car data. Below i explained in detail how to develop an SLR model.
# We'll use libraries like numpy for numerical operations, pandas for data manipulation,
# matplotlib for plotting, and sklearn for machine learning tasks.

import numpy as np  # Importing numpy for numerical operations
import pandas as pd  # Importing pandas for data manipulation
import matplotlib.pyplot as plt  # Importing matplotlib for plotting
from sklearn.linear_model import LinearRegression  # Importing LinearRegression from sklearn
from sklearn.metrics import mean_squared_error  # Importing mean_squared_error from sklearn
from sklearn.model_selection import train_test_split  # Importing train_test_split from sklearn

# We'll ignore any warnings to keep our output clean.
import warnings
warnings.filterwarnings('ignore')  # This will suppress any warning messages
```

In [12]:
```python
# Let's start by loading our dataset into a pandas DataFrame.
car_data = pd.read_csv('fuel_efficiency_data.csv')  # Reading the CSV file into a DataFrame

# Displaying the last 7 rows of the dataset to get an idea of what it looks like.
print(car_data.tail(7))  # Displaying the last 7 rows of the DataFrame

# Displaying statistical summary of the dataset.
print(car_data.describe())  # Showing statistical summary like mean, standard deviation, etc.
```

```
    Engine_Size_Liters  Fuel_Efficiency_MPG
93            4.757576            26.483313
94            4.797980            24.691054
95            4.838384            24.029591
96            4.878788            26.206605
97            4.919192            26.501588
98            4.959596            23.633990
99            5.000000            23.923769
       Engine_Size_Liters  Fuel_Efficiency_MPG
count          100.000000           100.000000
mean             3.000000            35.116328
std              1.172181             6.290352
min              1.000000            22.505760
25%              2.000000            29.886659
50%              3.000000            35.666563
75%              4.000000            39.902106
max              5.000000            45.990208
```

In [13]:
```python
# Separating the independent variable (Engine Size in Liters) and dependent variable (Fuel Efficiency in MPG).
indp_vars = car_data[['Engine_Size_Liters']]  # Selecting the 'Engine_Size_Liters' column as independent variable
dep_var = car_data['Fuel_Efficiency_MPG']  # Selecting the 'Fuel_Efficiency_MPG' column as dependent variable
```

In [14]:
```python
# Plotting the relationship between Engine Size and Fuel Efficiency. We need to make sure that the indpendant and dependent varaibles are linearly corelated.
plt.scatter(indp_vars, dep_var)  # Creating a scatter plot with Engine Size on x-axis and Fuel Efficiency on y-axis
plt.xlabel("Engine Size (L)")  # Labeling the x-axis
plt.ylabel("Fuel Efficiency (MPG)")  # Labeling the y-axis
plt.title("Engine Size vs Fuel Efficiency")  # Adding a title to the plot
plt.show()  # Displaying the plot
```

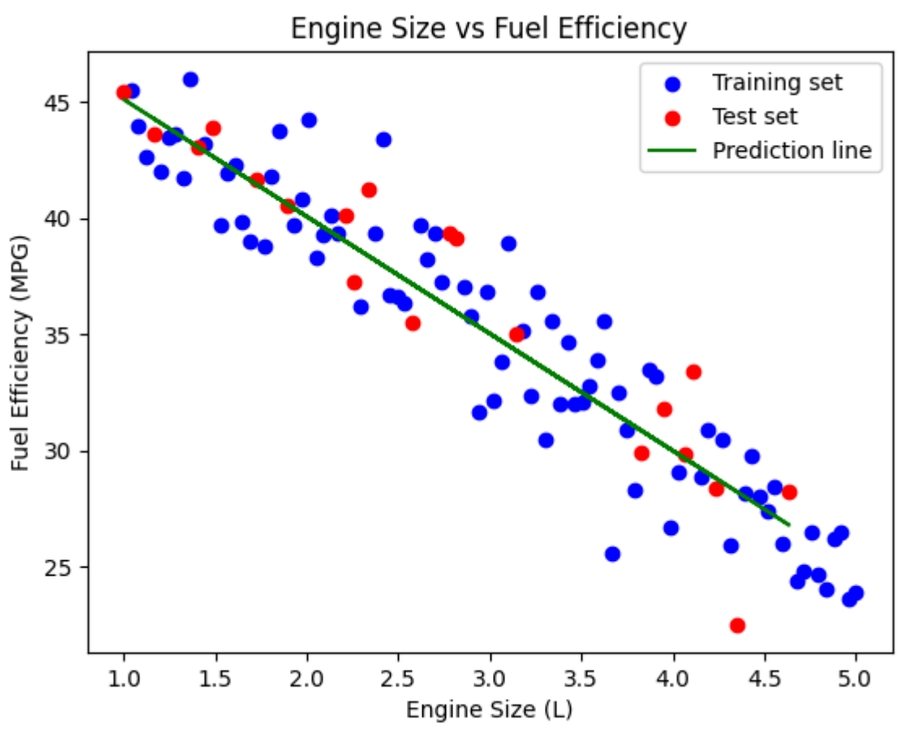


In [15]:
```python
# Splitting the data into training and testing sets.
train_x, test_x, train_y, test_y = train_test_split(indp_vars, dep_var, test_size=0.2, random_state=42)  # Splitting the data with 80% for training and 20% for testing

# Creating a Linear Regression model and fitting it to the training data.
lr_model = LinearRegression()  # Creating an instance of LinearRegression
lr_model.fit(train_x, train_y)  # Fitting the model to the training data

# Using the model to predict Fuel Efficiency on the test data.
pred = lr_model.predict(test_x)  # Making predictions on the test data
```

In [16]:
```python
# Calculating and printing the Mean Squared Error of the model's predictions.
mse = mean_squared_error(pred, test_y)  # Calculating the Mean Squared Error between predicted and actual values
print("Mean Squared Error:", mse)  # Printing the Mean Squared Error
```

```
Mean Squared Error: 4.570596048538351
```

In [17]:
```python
# Plotting the training set, test set, and the model's prediction line.
plt.scatter(train_x, train_y, color='blue', label='Training set')  # Plotting the training data points in blue
plt.scatter(test_x, test_y, color='red', label='Test set')  # Plotting the test data points in red
plt.plot(test_x, pred, color="green", label="Prediction line")  # Plotting the prediction line in green
plt.legend()  # Adding a legend to distinguish between training, test data, and prediction line
plt.xlabel("Engine Size (L)")  # Labeling the x-axis
plt.ylabel("Fuel Efficiency (MPG)")  # Labeling the y-axis
plt.title("Engine Size vs Fuel Efficiency")  # Adding a title to the plot
plt.show()  # Displaying the plot
```



In [18]:
```
# Asking the user to input a new engine size to predict its fuel efficiency.
new_entry = input("Please enter an engine size in liters: ")  # Prompting the user to enter an engine size
new_entry = np.array([[float(new_entry)]])  # Converting the user input to a float and then to a numpy array
pred_new = lr_model.predict(new_entry)  # Using the model to predict the fuel efficiency for the new engine size
print("Predicted Fuel Efficiency (MPG):", pred_new)  # Printing the predicted fuel efficiency
```

```
Predicted Fuel Efficiency (MPG): [40.06152765]
```

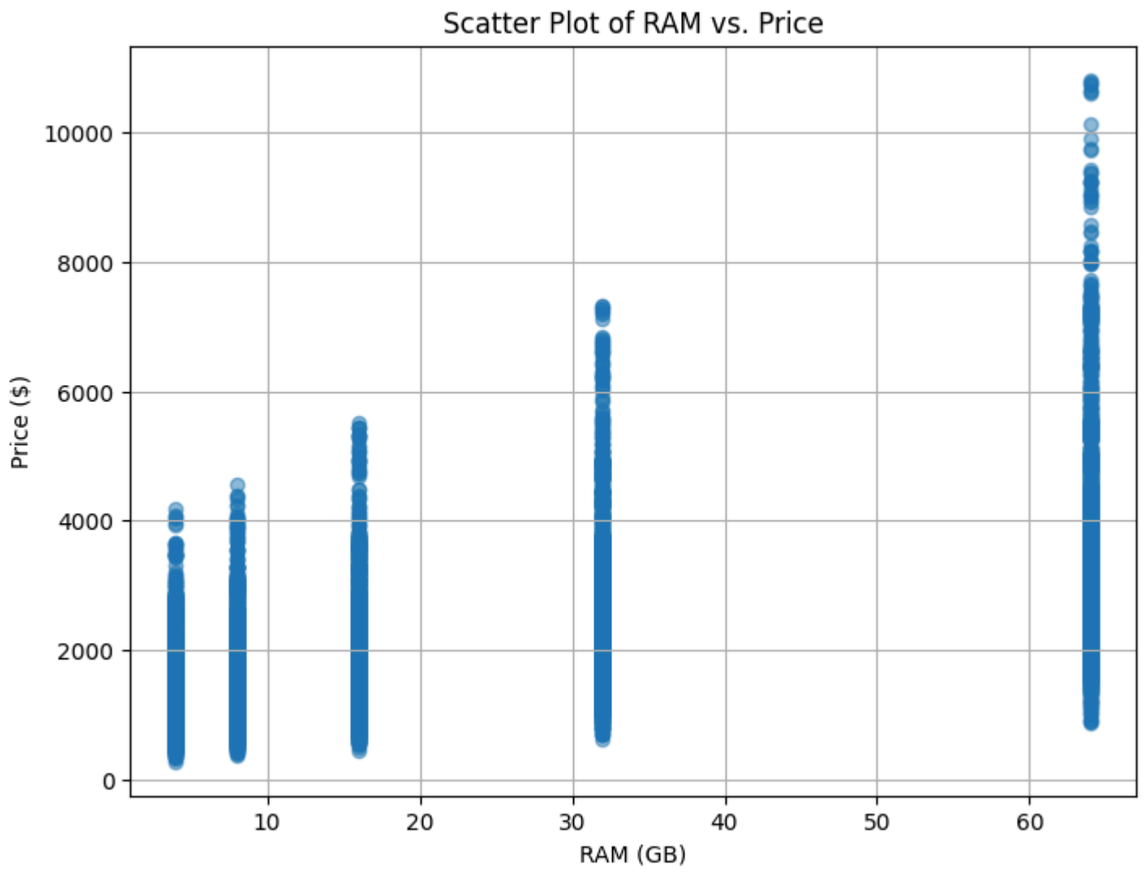
Create a Categorical Plot for the column Sex of the Titanic dataset.

In [20]:
```
df = pd.read_csv("laptop_prices.csv")
df.info(), df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11768 entries, 0 to 11767
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Brand                 11768 non-null  object 
 1   Processor             11768 non-null  object 
 2   RAM (GB)              11768 non-null  int64  
 3   Storage               11768 non-null  object 
 4   GPU                   11768 non-null  object 
 5   Screen Size (inch)    11768 non-null  float64
 6   Resolution            11768 non-null  object 
 7   Battery Life (hours)  11768 non-null  float64
 8   Weight (kg)           11768 non-null  float64
 9   Operating System      11768 non-null  object 
 10  Price ($)             11768 non-null  float64
dtypes: float64(4), int64(1), object(6)
memory usage: 1011.4+ KB
```

Out[20]:
```
(None,
     Brand    Processor  RAM (GB)    Storage                GPU  \
  0   Apple  AMD Ryzen 3        64  512GB SSD     Nvidia GTX 1650   
  1   Razer  AMD Ryzen 7         4    1TB SSD     Nvidia RTX 3080   
  2    Asus     Intel i5        32    2TB SSD     Nvidia RTX 3060   
  3  Lenovo     Intel i5         4  256GB SSD     Nvidia RTX 3080   
  4   Razer     Intel i3         4  256GB SSD  AMD Radeon RX 6600   

   Screen Size (inch) Resolution  Battery Life (hours)  Weight (kg)  \
0                17.3  2560x1440                   8.9         1.42   
1                14.0   1366x768                   9.4         2.57   
2                13.3  3840x2160                   8.5         1.74   
3                13.3   1366x768                  10.5         3.10   
4                16.0  3840x2160                   5.7         3.38   

  Operating System  Price ($)  
0          FreeDOS    3997.07  
1            Linux    1355.78  
2          FreeDOS    2673.07  
3          Windows     751.17  
4            Linux    2059.83  )
```

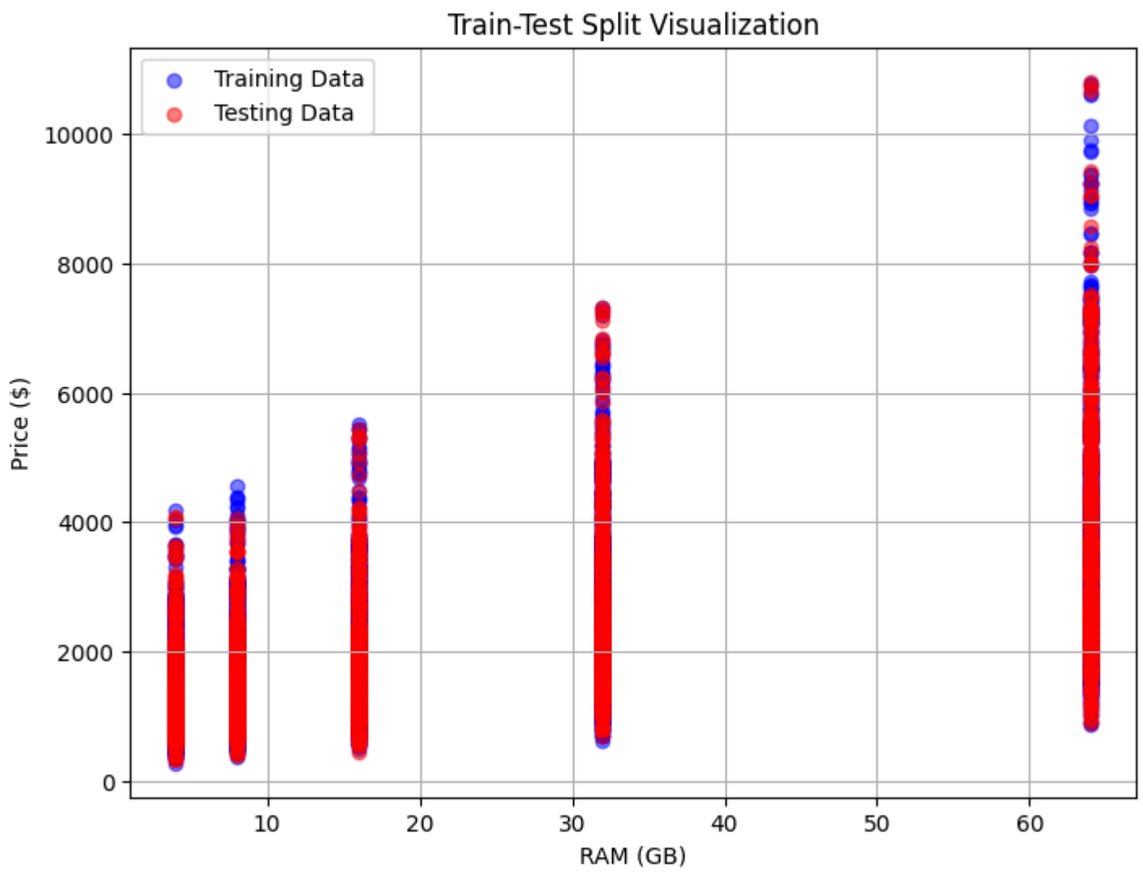
In [21]:
```
plt.figure(figsize=(8, 6))
plt.scatter(df["RAM (GB)"], df["Price ($)"], alpha=0.5)
plt.xlabel("RAM (GB)")
plt.ylabel("Price ($)")
plt.title("Scatter Plot of RAM vs. Price")
plt.grid(True)
plt.show()
```



In [23]:
```
X = df[["RAM (GB)"]]
y = df["Price ($)"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"Training set: X_train = {X_train.shape}, y_train = {y_train.shape}")
print(f"Testing set: X_test = {X_test.shape}, y_test = {y_test.shape}")
```

```
Training set: X_train = (8237, 1), y_train = (8237,)
Testing set: X_test = (3531, 1), y_test = (3531,)
```

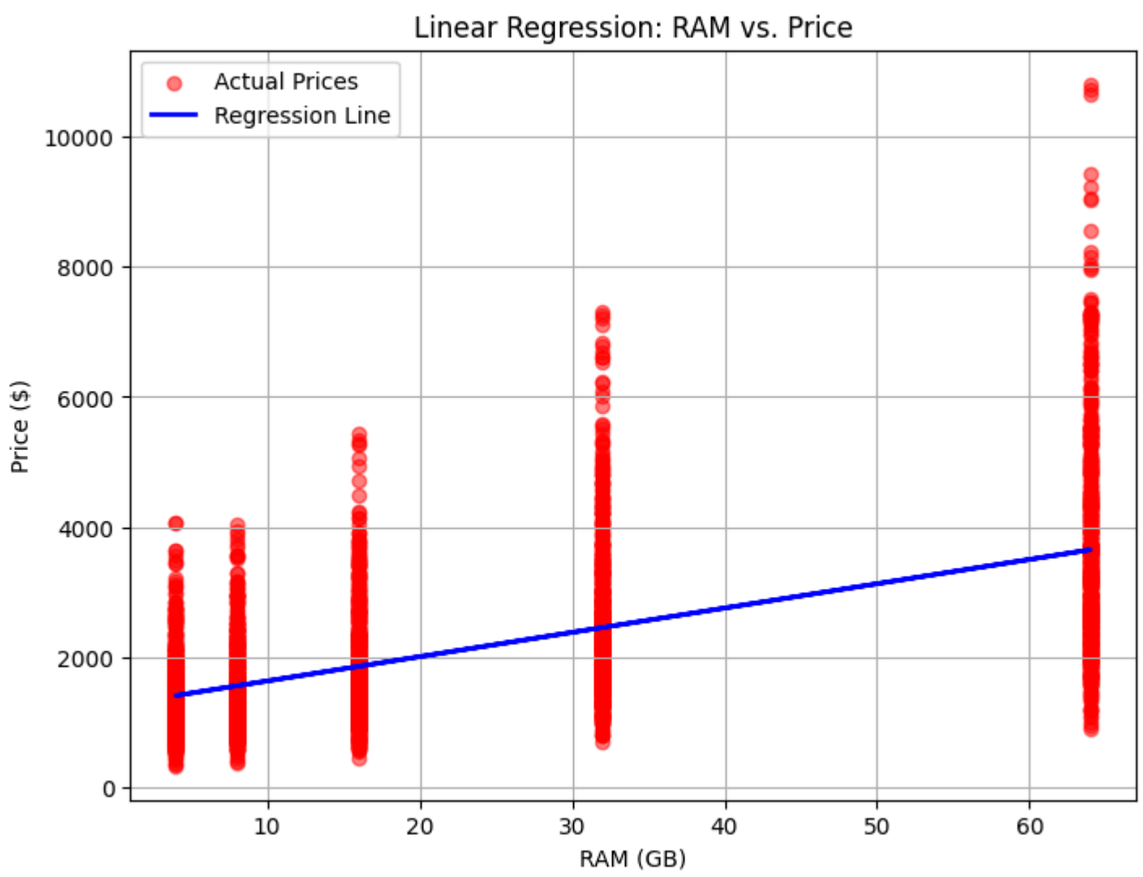
In [24]:
```
plt.figure(figsize=(8, 6))
plt.scatter(X_train, y_train, color="blue", alpha=0.5, label="Training Data")
plt.scatter(X_test, y_test, color="red", alpha=0.5, label="Testing Data")
plt.xlabel("RAM (GB)")
plt.ylabel("Price ($)")
plt.title("Train-Test Split Visualization")
plt.legend()
plt.grid(True)
plt.show()
```



In [27]:
```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = model.score(X_test, y_test)
print(f"Model Coefficients: {model.coef_[0]}")
print(f"Model Intercept: {model.intercept_}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared (R2) Score: {r2:.4f}")
```

```
Model Coefficients: 37.34964425425711
Model Intercept: 1262.177605967165
Mean Squared Error (MSE): 1118445.05
R-squared (R2) Score: 0.3781
```

In [28]:
```
plt.figure(figsize=(8, 6))
plt.scatter(X_test, y_test, color="red", alpha=0.5, label="Actual Prices")
plt.plot(X_test, y_pred, color="blue", linewidth=2, label="Regression Line")  # Regression line
plt.xlabel("RAM (GB)")
plt.ylabel("Price ($)")
plt.title("Linear Regression: RAM vs. Price")
plt.legend()
plt.grid(True)
plt.show()
```



In [ ]: